

# IMPLEMENTATION TASK PLAN — Reactor Rush v1.0

---

## WAVE 1 — Project Infrastructure

---

### TASK-01 · Root package and monorepo layout

**Objective:** Create the repository skeleton, root `package.json` with workspaces, `.gitignore`, and `.env.example`. No logic yet.

**Files to create:**

```
package.json
.gitignore
.env.example
backend/      (empty — scaffold only)
frontend/    (empty — scaffold only)
docs/        (empty — scaffold only)
tests/e2e/   (empty — scaffold only)
.github/workflows/ (empty — scaffold only)
```

**Dependencies:** none.

**Exact changes:**

`package.json` (root):

```
{
  "name": "reactor-rush",
  "private": true,
  "workspaces": ["backend", "frontend"],
  "scripts": {
    "build": "npm run build --workspace=backend && npm run build --workspace=frontend",
    "start": "node backend/dist/index.js",
    "lint": "npm run lint --workspace=backend && npm run lint --workspace=frontend",
    "test:e2e": "playwright test",
    "test:e2e:headed": "playwright test --headed"
  }
}
```

`.gitignore: node_modules/, dist/, .env, playwright-report/, test-results/.`

`.env.example:`

```
PORT=3000
NODE_ENV=development
VITE_API_BASE=/api
PRODUCTION_URL=
TEST_ENV=
```

#### Acceptance criteria:

- `npm install` at root resolves without errors.
- Directory scaffold matches the approved design.
- No business logic exists yet.

#### Validation:

```
npm install
ls backend frontend docs tests .github
```

**Expected result:** monorepo skeleton ready for individual package setup.

---

## TASK-02 · Backend package setup (Express + TypeScript)

**Objective:** `backend/package.json` and `backend/tsconfig.json` configured. No prohibited libraries.

#### Files to create:

```
backend/package.json
backend/tsconfig.json
backend/src/          (empty)
backend/src/game/     (empty)
backend/src/routes/   (empty)
```

**Dependencies:** TASK-01.

#### Exact changes:

`backend/package.json` — dependencies: `express`, `@types/express`, `@types/node`, `typescript`. devDependencies: `ts-node-dev`, `eslint`, `@typescript-eslint/parser`, `@typescript-eslint/eslint-plugin`. Scripts:

```
build: tsc, dev: ts-node-dev --respawn src/index.ts, start: node
dist/index.js, lint: eslint src --ext .ts.
```

No `axios`, no game engine libraries, no ORM, no `socket.io`.

`backend/tsconfig.json`:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "outDir": "dist",
    "rootDir": "src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  }
}
```

#### Acceptance criteria:

- `cd backend && npm install` succeeds.
- No prohibited package appears in `node_modules`.
- `grep -E "axios|socket|redux|sequelize|mongoose" backend/package.json` returns empty.

#### Validation:

```
cd backend && npm install
cd backend && npx tsc --version
grep -E "axios|socket|redux" backend/package.json
```

**Expected result:** backend can compile TypeScript; no prohibited libraries installed.

---

## TASK-03 · Backend minimal entry point

**Objective:** `backend/src/index.ts` starts an Express server with a `/health` endpoint. Compiles and runs.

#### Files to create:

`backend/src/index.ts`

**Dependencies:** TASK-02.

**Exact changes:**

backend/src/index.ts:

- Create Express app.
- Register `express.json()` middleware.
- Register `GET /health` returning `{ status: 'ok' }`.
- Listen on `process.env.PORT ?? 3000`.
- `express.static` and SPA fallback will be added in TASK-32 (after frontend build exists).

**Acceptance criteria:**

- `cd backend && npm run build` exits 0.
- `node backend/dist/index.js` & then `curl http://localhost:3000/health` returns `{"status": "ok"}`.
- `npx tsc --noEmit` exits 0.

**Validation:**

```
cd backend && npm run build
node backend/dist/index.js &
sleep 1
curl -s http://localhost:3000/health
kill %1
```

**Expected result:** backend starts and responds to HTTP requests.

---

## TASK-04 · Frontend package setup (React + TypeScript + Vite)

**Objective:** `frontend/package.json`, `tsconfig.json`, and `vite.config.ts` configured. No prohibited libraries.

**Files to create:**

```
frontend/package.json
frontend/tsconfig.json
frontend/vite.config.ts
frontend/index.html
frontend/src/          (empty)
frontend/src/components/ (empty)
frontend/src/hooks/    (empty)
frontend/src/api/      (empty)
```

**Dependencies:** TASK-01.

**Exact changes:**

frontend/package.json — dependencies: react, react-dom. devDependencies: typescript, @types/react, @types/react-dom, vite, @vitejs/plugin-react, eslint, @typescript-eslint/parser, @typescript-eslint/eslint-plugin, eslint-plugin-react-hooks. Scripts: build: vite build, dev: vite, lint: eslint src --ext .ts, .tsx, typecheck: tsc --noEmit.

No axios, react-router, redux, tailwind, bootstrap, styled-components, game engines.

frontend/tsconfig.json: strict, jsx react-jsx, module ESNext, target ESNext.

frontend/vite.config.ts:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
```

```
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': 'http://localhost:3000'
    }
  }
});
```

frontend/index.html: HTML5 with <div id="root"> and <script type="module" src="/src/main.tsx">.

**Acceptance criteria:**

- cd frontend && npm install succeeds.
- grep -E "axios|react-router|redux|tailwind|bootstrap" frontend/package.json returns empty.
- frontend/tsconfig.json has "strict": true.

**Validation:**

```
cd frontend && npm install
grep -E "axios|react-router|redux|tailwind|bootstrap" frontend/package.json
```

**Expected result:** frontend environment ready with no prohibited libraries.

---

## TASK-05 · Frontend minimal React entry point

**Objective:** Frontend compiles to `frontend/dist/` with a placeholder App.

**Files to create:**

`frontend/src/main.tsx`  
`frontend/src/App.tsx`

**Dependencies:** TASK-04.

**Exact changes:**

`frontend/src/main.tsx`:  
`ReactDOM.createRoot(document.getElementById('root')).render(<App />).`

`frontend/src/App.tsx`: returns `<h1>Reactor Rush</h1>` (placeholder; replaced in Wave 9).

**Acceptance criteria:**

- `cd frontend && npm run build` exits 0 and produces `frontend/dist/index.html`.
- `cd frontend && npm run typecheck` exits 0.

**Validation:**

`cd frontend && npm run build`  
`ls frontend/dist/index.html`  
`cd frontend && npm run typecheck`

**Expected result:** frontend produces a deployable static build.

---

## TASK-06 · ESLint configuration for backend and frontend

**Objective:** ESLint runs on both packages with TypeScript-strict rules. No `any`, no unused vars.

**Files to create:**

backend/.eslintrc.json  
frontend/.eslintrc.json

**Dependencies:** TASK-02, TASK-04.

**Exact changes:**

backend/.eslintrc.json:

```
{
  "parser": "@typescript-eslint/parser",
  "plugins": ["@typescript-eslint"],
  "extends": ["eslint:recommended", "plugin:@typescript-eslint/recommended"],
  "rules": {
    "@typescript-eslint/no-explicit-any": "error",
    "no-console": "warn"
  }
}
```

frontend/.eslintrc.json: same base rules plus  
"plugin:react-hooks/recommended".

Add "lint" script to both package.json files (already added in TASK-02 and TASK-04).

**Acceptance criteria:**

- npm run lint --workspace=backend exits 0 on empty source files.
- npm run lint --workspace=frontend exits 0 on placeholder source.
- Introducing const x: any = 1 causes lint failure.

**Validation:**

```
npm run lint --workspace=backend
npm run lint --workspace=frontend
```

**Expected result:** linting enforces TypeScript quality on all source files.

---

## WAVE 2 — Shared / Domain Types

---

### TASK-07 · Backend game types

**Objective:** Define every type needed by the game in `backend/src/game/state.ts`. This is the authoritative source of truth.

**Files to create:**

`backend/src/game/state.ts`

**Dependencies:** TASK-02.

**Exact changes — define and export all of the following:**

```
export type CellType = 'empty' | 'wall' | 'destructible';
export type GameStatus = 'playing' | 'finished';
export type GameResult = 'player1_wins' | 'player2_wins' | 'draw' | null;
export type PlayerId = 'player1' | 'player2';
export type ActionType = 'move' | 'place_bomb' | 'capture_core' | 'special_action';
export type Direction = 'up' | 'down' | 'left' | 'right';
export type ResourceType = 'energy_pack' | 'repair_kit';
```

```
export interface PlayerState {
  id: PlayerId;
  x: number; y: number;
  hp: number;      // 0–3
  energy: number;  // 0–10
  resources: number; // collected count
  score: number;
  bombAvailable: boolean;
  shieldActive: boolean;
  alive: boolean;
}
```

```
export interface BombState {
  id: string;
  playerId: PlayerId;
  x: number; y: number;
  timerRemaining: number; // seconds, for visual only
}
```

```
export interface CoreState { id: string; x: number; y: number; }
```

```
export interface ResourceState {
  id: string;
  x: number; y: number;
  type: ResourceType;
}
```

```
export interface GameState {
```

```

gameId: string;
status: GameStatus;
timeRemaining: number;
result: GameResult;
players: { player1: PlayerState; player2: PlayerState };
bombs: BombState[];
cores: CoreState[];
resources: ResourceState[];
grid: CellType[][]; // grid[y][x], 11 rows × 15 columns
arenaEventActive: boolean;
arenaEventCountdown: number | null;
}

```

```

export interface ActionRequest {
  playerId: PlayerId;
  type: ActionType;
  payload?: { direction?: Direction };
}

```

```

export class GameActionError extends Error {
  constructor(
    public readonly code: string,
    public readonly message: string,
    public readonly httpStatus: number
  ) { super(message); }
}

```

### Acceptance criteria:

- `cd backend && npx tsc --noEmit` exits 0.
- No `any` anywhere in the file.
- Every field from the approved `GameState` design is present.

### Validation:

```

cd backend && npx tsc --noEmit
cd backend && npm run lint

```

**Expected result:** complete type contract for backend game logic.

## TASK-08 · Frontend game types (mirrored)

**Objective:** Replicate all necessary types in `frontend/src/types.ts`. No shared module between packages — types are kept in sync manually.

**Files to create:**

frontend/src/types.ts

**Dependencies:** TASK-07 (read as reference; do not import from backend).

**Exact changes:** Copy every type from `backend/src/game/state.ts` into `frontend/src/types.ts`. Additionally add:

```
export type ActionResponse =  
  | { success: true; state: GameState }  
  | { success: false; error: string; message: string };  
  
export interface CreateGameResponse {  
  gameId: string;  
  state: GameState;  
}
```

**Acceptance criteria:**

- `cd frontend && npm run typecheck` exits 0.
- No `import` from `../.. / backend` in the file.
- No `any` in the file.

**Validation:**

```
cd frontend && npm run typecheck  
cd frontend && npm run lint  
grep "from.*backend" frontend/src/types.ts  
# must return empty
```

**Expected result:** frontend has a complete type contract independent of backend source files.

---

## WAVE 3 — Backend Store and Game Factory

---

**TASK-09 · Game constants file**

**Objective:** All configurable game numbers live in one file. The teacher can change any constant and every module picks it up automatically.

**Files to create:**

backend/src/game/constants.ts

**Dependencies:** TASK-07.

**Exact changes:**

```
export const GRID_COLS = 15;
export const GRID_ROWS = 11;

export const P1_SPAWN = { x: 1, y: 1 };
export const P2_SPAWN = { x: 13, y: 9 };

export const INITIAL_HP = 3;
export const MAX_HP = 3;
export const INITIAL_ENERGY = 5;
export const MAX_ENERGY = 10;

export const GAME_DURATION_S = 120;
export const VICTORY_SCORE = 25;

export const BOMB_TIMER_S = 3;
export const BOMB_RADIUS = 2;

export const SHIELD_DURATION_MS = 4_000;

export const SCORE_CAPTURE_CORE = 5;
export const SCORE_KILL = 10;
export const SCORE_PICKUP_RESOURCE = 1;
export const SCORE_SPECIAL_HIT = 2;

export const ENERGY_CAPTURE_CORE = 2;
export const ENERGY_SPECIAL_COST = 3;
export const ENERGY_SPECIAL_HIT_DRAIN = 2;

export const ENERGY_PACK_REGEN = 2;
export const REPAIR_KIT_REGEN = 1;

export const CORE_RESPAWN_MS = 8_000;
export const INITIAL_CORE_COUNT = 3;
export const MAX_CORE_COUNT = 3;
export const INITIAL_RESOURCE_COUNT = 4;

export const DESTRUCTIBLE_MIN = 15;
export const DESTRUCTIBLE_MAX = 25;

export const REACTOR_PULSE_MIN_S = 45;
export const REACTOR_PULSE_MAX_S = 75;
```

```
export const REACTOR_PULSE_WARNING_S = 5;
export const RESOURCE_SPAWN_CHANCE = 0.5;

export const CHAIN_REACTION_MAX_DEPTH = 5;
export const POLLING_INTERVAL_MS = 500; // informational — used in frontend
```

### Acceptance criteria:

- `cd backend && npx tsc --noEmit` exits 0.
- No magic numbers appear in any other backend module after all waves are complete (verified in TASK-57).

### Validation:

```
cd backend && npx tsc --noEmit
```

**Expected result:** single authoritative location for every tunable game constant.

---

## TASK-10 · In-memory store and timer registry

**Objective:** Implement `backend/src/store.ts` — single-game in-memory store with safe timer management.

### Files to create:

```
backend/src/store.ts
```

**Dependencies:** TASK-07.

### Exact changes:

```
interface GameTimerEntry {
  interval:    NodeJS.Timeout | null;
  timeouts:    NodeJS.Timeout[];
  respawningCores: Set<string>; // coreIds currently scheduled for respawn
}
```

```
// Singletons
```

```
const store = new Map<string, GameState>();
const timers = new Map<string, GameTimerEntry>();
let currentGameId: string | null = null;
```

```
export function getGame(id: string): GameState | undefined
export function setGame(id: string, state: GameState): void
```

```

export function deleteGame(id: string): void
export function getCurrentGameId(): string | null
export function setCurrentGameId(id: string | null): void

export function initGameTimers(gameId: string): void
  // creates fresh GameTimerEntry for gameId

export function getGameTimers(gameId: string): GameTimerEntry | undefined

export function clearGameTimers(gameId: string): void
  // 1. clearInterval on .interval
  // 2. clearTimeout on each entry in .timeouts
  // 3. clear .respawningCores
  // 4. delete from timers map
  // Does NOT throw if gameId is unknown.

```

### Acceptance criteria:

- `setGame` then `getGame` returns the same reference.
- `clearGameTimers` called twice on the same id does not throw.
- `clearGameTimers` on an unknown id does not throw.
- `cd backend && npx tsc --noEmit` exits 0.

### Validation:

```

cd backend && npx tsc --noEmit
cd backend && npm run lint

```

**Expected result:** safe, testable in-memory storage with no timer leaks.

## TASK-11 · Grid generation — walls and pattern

**Objective:** `generateGrid()` produces a 15×11 grid with border walls and fixed internal wall pattern.

### Files to create:

backend/src/game/factory.ts (partial — grid only)

**Dependencies:** TASK-07, TASK-09.

### Exact changes:

Export `generateGrid(destructibleOverride?: Array<{x:number,y:number}>): CellType[][]`:

1. Initialise `grid[y][x]` as `'empty'` for all 11×15 positions.
2. Set entire border (row 0, row 10, col 0, col 14) to `'wall'`.
3. Set interior positions where `x % 2 === 0 && y % 2 === 0` and `x ∈ [2,12], y ∈ [2,8]` to `'wall'`.
4. If `destructibleOverride` provided: mark each position as `'destructible'` and return.
5. Otherwise: continue (random destructibles added in TASK-12).

#### Acceptance criteria:

- Returned grid has exactly 11 rows, each with 15 elements.
- All border cells are `'wall'`.
- Interior positions `(2,2)`, `(4,2)`, `(2,4)` etc. are `'wall'`.
- `(1,1)` and `(13,9)` are `'empty'`.
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

`cd backend && npx tsc --noEmit`

**Expected result:** deterministic grid skeleton with no destructibles yet.

---

## TASK-12 · BFS utilities and reachability validation

**Objective:** Implement `bfsReachable` and `getReachableCells` used by the factory for connectivity guarantees.

#### Files to create:

`backend/src/game/bfs.ts`

**Dependencies:** TASK-07, TASK-09.

#### Exact changes:

```
export function bfsReachable(
  grid: CellType[][],
  from: {x:number,y:number},
  to: {x:number,y:number}
): boolean
// Returns true if a path exists traversing only non-wall, non-destructible cells.
```

```
export function getReachableCells(  
  grid: CellType[][],  
  from: {x:number,y:number}  
): Set<string>  
// Returns Set of "x,y" keys for all cells reachable from `from`.
```

Both use standard BFS over the 4 cardinal directions.

#### Acceptance criteria:

- On a fully open grid, both spawns are reachable from each other.
- On a grid with a complete blocking wall row, `bfsReachable` returns false.
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

```
cd backend && npx tsc --noEmit  
cd backend && npm run lint
```

**Expected result:** standalone BFS module, importable by any backend module that needs pathfinding.

---

## TASK-13 · Random destructibles with safety zones and BFS fix-up

**Objective:** Complete `generateGrid()` to place random destructibles while keeping spawns connected and spawn safety zones clear.

#### Files to modify:

backend/src/game/factory.ts

**Dependencies:** TASK-11, TASK-12.

#### Exact changes:

Continue `generateGrid()` after step 3 (when no `destructibleOverride`):

4. Compute safety zone: the 4 orthogonal neighbours of `P1_SPAWN` and `P2_SPAWN` plus the spawns themselves.
5. Collect candidate cells: all interior `'empty'` cells not in safety zone.
6. Shuffle candidates. Pick `N = randomInt(DESTRUCTIBLE_MIN, DESTRUCTIBLE_MAX)` of them. Mark as `'destructible'`.
7. Run `bfsReachable(grid, P1_SPAWN, P2_SPAWN)`. If false:

- Find and remove destructibles that lie on or near the blocked corridor (iterate destructibles closest to the midpoint first).
  - Repeat BFS until path exists. Maximum 50 removal attempts; if still blocked after 50, clear all destructibles (safe fallback).
8. Return grid.

Helper `randomInt(min: number, max: number): number` — inclusive both ends.

#### Acceptance criteria:

- After 100 calls, every returned grid passes `bfsReachable(P1_SPAWN, P2_SPAWN)`.
- Safety zones at `(1,1)` and `(13,9)` never contain `'destructible'`.
- Destructible count is between `DESTRUCTIBLE_MIN` and `DESTRUCTIBLE_MAX` (may be lower after fix-up).
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

```
cd backend && npx tsc --noEmit
# scripted stress test: run generateGrid() 100 times, assert BFS passes each time
```

**Expected result:** every generated grid is playable with guaranteed connectivity.

## TASK-14 · Core and resource placement

**Objective:** `createGame()` places cores and resources only on reachable `'empty'` cells.

#### Files to modify:

`backend/src/game/factory.ts`

**Dependencies:** TASK-13.

#### Exact changes:

```
Export interface TestScenario { destructibles: Array<{x,y}>; cores:
Array<{x,y}>; resources: Array<{x,y,type:ResourceType}> }.
```

```
Export createGame(scenario?: TestScenario): GameState:
```

1. `gameId = crypto.randomUUID()`.
2. `grid = generateGrid(scenario?.destructibles)`.
3. `reachable = getReachableCells(grid, P1_SPAWN)`.

4. Filter reachable to exclude spawns and any cell already used.
5. **Cores:** if `scenario?.cores`: use those positions; else pick `INITIAL_CORE_COUNT` random cells from reachable.
6. **Resources:** if `scenario?.resources`: use those; else pick `INITIAL_RESOURCE_COUNT` random cells from remaining reachable. Assign type randomly (50/50).
7. Build full `GameState` with values from `constants.ts`:
  - `player1: { id: 'player1', x: 1, y: 1, hp: INITIAL_HP, energy: INITIAL_ENERGY, resources: 0, score: 0, bombAvailable: true, shieldActive: false, alive: true }.`
  - `player2`: same structure at (13, 9).
  - `status: 'playing', timeRemaining: GAME_DURATION_S, result: null, arenaEventActive: false, arenaEventCountdown: null.`
8. `setGame(gameId, state), setCurrentGameId(gameId), initGameTimers(gameId).`
9. Return `GameState`. (Timers started separately by TASK-21 `startGameLoop`.)

#### Acceptance criteria:

- All cores land on 'empty' reachable cells.
- All resources land on 'empty' reachable cells distinct from cores.
- Spawns are not used for cores or resources.
- With `scenario = { destructibles: [], cores: [{x: 3, y: 1}], resources: [] }:` `state.cores[0]` is `{x: 3, y: 1}`, no destructibles.
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

```
cd backend && npx tsc --noEmit
cd backend && npm run lint
```

**Expected result:** a complete, valid `GameState` is constructable from `createGame()`.

---

## TASK-15 · Timer cleanup on game restart

**Objective:** Calling `createGame()` while a game is already active cancels all previous timers before creating the new game.

#### Files to modify:

```
backend/src/game/factory.ts
backend/src/store.ts
```

**Dependencies:** TASK-14, TASK-10.

**Exact changes:**

At the **start** of `createGame()`, before generating anything:

```
const existing = getCurrentGameId();
if (existing !== null) {
  clearGameTimers(existing); // cancel all timers
  deleteGame(existing);      // remove state
  setCurrentGameId(null);
}
```

**Acceptance criteria:**

- After `createGame()` is called twice, `GET /api/game/:oldId` returns 404.
- `timeRemaining` of the first game stops decrementing after the second call (manual timing test, verified in TASK-41 integration check).
- `cd backend && npx tsc --noEmit` exits 0.

**Validation:**

```
cd backend && npx tsc --noEmit
```

**Expected result:** no timer leaks on game restart.

---

## WAVE 4 — Backend Game Engine

---

### TASK-16 · `checkVictory` function

**Objective:** Deterministic victory/draw detection that mutates `GameState.status` and `GameState.result`.

**Files to create:**

```
backend/src/game/engine.ts (partial)
```

**Dependencies:** TASK-07, TASK-09.

**Exact changes:**

```
export function checkVictory(state: GameState): void {
  if (state.status !== 'playing') return; // idempotent
  const s1 = state.players.player1.score;
  const s2 = state.players.player2.score;
  if (s1 >= VICTORY_SCORE) {
    state.status = 'finished'; state.result = 'player1_wins'; return;
  }
  if (s2 >= VICTORY_SCORE) {
    state.status = 'finished'; state.result = 'player2_wins'; return;
  }
  if (state.timeRemaining <= 0) {
    state.status = 'finished';
    state.result = s1 > s2 ? 'player1_wins' : s2 > s1 ? 'player2_wins' : 'draw';
  }
}
```

#### Acceptance criteria:

- Score 25/10, time 60 → **player1\_wins**.
- Score 10/25, time 60 → **player2\_wins**.
- Score 10/10, time 0 → **draw**.
- Score 15/10, time 0 → **player1\_wins**.
- Calling twice with same state does not flip result.
- **cd backend && npx tsc --noEmit** exits 0.

#### Validation:

**cd backend && npx tsc --noEmit**

**Expected result:** deterministic, idempotent victory check.

---

## TASK-17 · **applyDamage** function

**Objective:** Centralised damage function with shield interaction, death detection, and kill score.

#### Files to modify:

backend/src/game/engine.ts

**Dependencies:** TASK-16.

#### Exact changes:

```

export function applyDamage(
  state:    GameState,
  targetId: PlayerId,
  amount:   number,
  attackerId: PlayerId | null,
  bypassShield: boolean = false
): void {
  const target = state.players[targetId];
  if (!target.alive) return;           // already dead

  if (!bypassShield && target.shieldActive) {
    target.shieldActive = false;       // shield consumed
    return;
  }

  target.hp = Math.max(0, target.hp - amount);

  if (target.hp === 0) {
    target.alive = false;
    if (attackerId !== null && attackerId !== targetId) {
      state.players[attackerId].score += SCORE_KILL;
    }
    checkVictory(state);
  }
}

```

### Acceptance criteria:

- `alive:false` target: no effect, function returns.
- Shield active (no bypass): shield consumed, no hp loss.
- Shield active (bypass): hp decremented normally.
- `hp = 1` → damage 1 → `hp = 0, alive = false`, attacker `score += SCORE_KILL`.
- Self-damage (`attackerId === targetId`): no kill score added.
- Double call: second call is a no-op because `alive` is already false.
- `cd backend && npx tsc --noEmit` exits 0.

### Validation:

`cd backend && npx tsc --noEmit`

**Expected result:** all damage pathways are handled in one place.

---

## TASK-18 · Game loop — time countdown and tick

**Objective:** `startGameLoop` runs a 1-second interval that decrements time, updates visual timers, decrements `arenaEventCountdown`, and calls `checkVictory`.

**Files to modify:**

backend/src/game/engine.ts

**Dependencies:** TASK-16, TASK-10.

**Exact changes:**

```
export function startGameLoop(gameId: string): void {
  initGameTimers(gameId); // safe to call even if already initialised
  const entry = getGameTimers(gameId)!;

  entry.interval = setInterval(() => {
    const state = getGame(gameId);
    if (!state || state.status !== 'playing') {
      clearGameTimers(gameId);
      return;
    }

    // 1. Decrement time
    state.timeRemaining = Math.max(0, state.timeRemaining - 1);

    // 2. Decrement bomb visual timers
    for (const bomb of state.bombs) {
      bomb.timerRemaining = Math.max(0, bomb.timerRemaining - 1);
    }

    // 3. Decrement arena event countdown
    if (state.arenaEventCountdown !== null && state.arenaEventCountdown > 0) {
      state.arenaEventCountdown -= 1;
    }

    // 4. Victory check
    checkVictory(state);

    // 5. Persist
    setGame(gameId, state);

    // 6. Clean up if finished
    if (state.status === 'finished') {
      clearGameTimers(gameId);
    }
  }, 1000);
}
```

**Acceptance criteria:**

- After 3 real seconds, `state.timeRemaining` is `GAME_DURATION_S - 3`.
- When `checkVictory` sets `status = 'finished'`, the interval self-cancels.
- If `clearGameTimers` is called externally, the interval also stops (verified by `getGame` returning undefined).
- `cd backend && npx tsc --noEmit` exits 0.

**Validation:**

`cd backend && npx tsc --noEmit`

**Expected result:** server-authoritative clock with self-managing lifecycle.

---

**TASK-19 · Wire `startGameLoop` into `createGame`**

**Objective:** Calling `createGame()` automatically starts the game loop. Time begins immediately.

**Files to modify:**

`backend/src/game/factory.ts`

**Dependencies:** TASK-14, TASK-18.

**Exact changes:**

At the end of `createGame()`, after `initGameTimers(gameId)`, add:  
`startGameLoop(gameId);`

- 
- Also add call `scheduleReactorPulse(gameId)` (stub import for now; implemented in TASK-30).

**Acceptance criteria:**

- After `createGame()`, waiting 2 seconds then calling `getGame(gameId)` shows `timeRemaining < GAME_DURATION_S`.
- `cd backend && npx tsc --noEmit` exits 0.

**Validation:**

`cd backend && npx tsc --noEmit`

**Expected result:** the game clock starts automatically on game creation.

---

## WAVE 5 — Player Actions

---

### TASK-20 · Action dispatch infrastructure

**Objective:** Create `backend/src/game/actions.ts` with the `processAction` entry point, common validation, and `GameActionError` usage.

**Files to create:**

`backend/src/game/actions.ts`

**Dependencies:** TASK-07, TASK-10, TASK-16.

**Exact changes:**

```
export function processAction(
  gameId: string,
  req: ActionRequest
): GameState {
  const state = getGame(gameId);
  if (!state) throw new GameActionError('GAME_NOT_FOUND', 'Game not found.', 404);

  validateCommon(state, req.playerId);

  const player = state.players[req.playerId];

  switch (req.type) {
    case 'move':      processMove(state, player, req.payload?.direction!); break;
    case 'place_bomb': processPlaceBomb(state, player); break;
    case 'capture_core': processCaptureCore(state, player); break;
    case 'special_action': processSpecialAction(state, player); break;
    default:
      throw new GameActionError('UNKNOWN_ACTION', 'Unknown action type.', 400);
  }

  setGame(gameId, state);
  return state;
}

function validateCommon(state: GameState, playerId: PlayerId): void {
```

```

if (state.status !== 'playing')
  throw new GameActionError('GAME_NOT_PLAYING', 'La partida ya ha terminado.', 409);
if (!state.players[playerId].alive)
  throw new GameActionError('PLAYER_ELIMINATED', 'El jugador ha sido eliminado.',
400);
}

```

The four `process*` functions are implemented in subsequent tasks. Add stubs that throw `'NOT_IMPLEMENTED'` temporarily so the file compiles.

#### Acceptance criteria:

- `cd backend && npx tsc --noEmit` exits 0.
- `processAction` with a finished game state throws `GAME_NOT_PLAYING` with status 409.
- `processAction` with `alive:false` player throws `PLAYER_ELIMINATED`.

#### Validation:

```

cd backend && npx tsc --noEmit
cd backend && npm run lint

```

**Expected result:** action dispatch with validated error handling infrastructure.

---

## TASK-21 · Action: `move`

**Objective:** Implement movement including bounds check, wall/destructible block, occupancy check, and automatic resource pickup.

#### Files to modify:

backend/src/game/actions.ts

**Dependencies:** TASK-20.

**Exact changes — implement `processMove(state, player, direction)`:**

1. Compute `(nx, ny)` from direction using delta map.
2. Bounds check  $0 \leq nx \leq \text{GRID\_COLS}-1$ ,  $0 \leq ny \leq \text{GRID\_ROWS}-1 \rightarrow \text{MOVE\_OUT\_OF\_BOUNDS}$  (400).
3. Cell check: `grid[ny][nx] === 'wall' || grid[ny][nx] === 'destructible' → MOVE_BLOCKED` (400).
4. Occupancy check: rival is at `(nx, ny) → MOVE_OCCUPIED` (400).

5. Update `player.x = nx, player.y = ny`.
6. Check `state.resources` for a resource at `(nx, ny)`:
  - If found: apply effect, `+= SCORE_PICKUP_RESOURCE, player.resources += 1`, remove from array.
  - `energy_pack`: `player.energy = Math.min(MAX_ENERGY, player.energy + ENERGY_PACK_REGEN)`.
  - `repair_kit`: `player.hp = Math.min(MAX_HP, player.hp + REPAIR_KIT_REGEN)`.

#### Acceptance criteria:

- Move to 'empty' cell: player x/y updated.
- Move to 'wall': error `MOVE_BLOCKED`.
- Move to 'destructible': error `MOVE_BLOCKED`.
- Move to rival's cell: error `MOVE_OCCUPIED`.
- Move off-grid: error `MOVE_OUT_OF_BOUNDS`.
- Move onto resource: resource removed, stats updated, `+1 score`.
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

`cd backend && npx tsc --noEmit`

**Expected result:** movement with all validations and automatic resource pickup.

---

## TASK-22 · Action: `place_bomb`

**Objective:** Register a bomb in the game state and mark the player's bomb as unavailable.

#### Files to modify:

`backend/src/game/actions.ts`

`backend/src/game/bombs.ts` (stub — created here if not yet present)

**Dependencies:** TASK-20.

**Exact changes — implement `processPlaceBomb(state, player)`:**

1. If `!player.bombAvailable` → `BOMB_ALREADY_ACTIVE (400)`.
2. Create `BombState { id: crypto.randomUUID(), playerId: player.id, x: player.x, y: player.y, timerRemaining: BOMB_TIMER_S }`.
3. Push to `state.bombs`.

4. `player.bombAvailable = false`.
5. Call `scheduleBomb(state.gameId, bomb.id)` — imported from `bombs.ts` (stub in this task; full impl in TASK-24).

**Acceptance criteria:**

- After `place_bomb`: `state.bombs.length === 1, player.bombAvailable === false`.
- Second `place_bomb`: error `BOMB_ALREADY_ACTIVE`.
- Bomb has `timerRemaining === BOMB_TIMER_S`.
- `cd backend && npx tsc --noEmit` exits 0.

**Validation:**

`cd backend && npx tsc --noEmit`

**Expected result:** bombs are registered in the server state.

---

## TASK-23 · Action: `capture_core`

**Objective:** Capture the core at the player's current cell, awarding score and energy.

**Files to modify:**

`backend/src/game/actions.ts`

`backend/src/game/cores.ts` (stub — created here if not yet present)

**Dependencies:** TASK-20, TASK-16.

**Exact changes — implement `processCaptureCore(state, player)`:**

1. Find `core` in `state.cores` where `x === player.x && y === player.y`.
2. If not found → `CORE_NOT_PRESENT` (400).
3. Remove core from `state.cores`.
4. `player.score += SCORE_CAPTURE_CORE`.
5. `player.energy = Math.min(MAX_ENERGY, player.energy + ENERGY_CAPTURE_CORE)`.
6. Call `scheduleRespawn(state.gameId, core.id, CORE_RESPAWN_MS)` — imported from `cores.ts` (stub).
7. Call `checkVictory(state)`.

**Acceptance criteria:**

- Player on core cell: score += 5, energy += 2 (capped at 10), core removed.
- Player NOT on core cell: error `CORE_NOT_PRESENT`.
- Capture bringing score to `VICTORY_SCORE`: `status === 'finished'`.
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

`cd backend && npx tsc --noEmit`

**Expected result:** core capture fully implemented with victory check.

---

## TASK-24 · Action: `special_action`

**Objective:** Implement the special action — offensive (adjacent rival) or defensive (shield) with cost validation.

#### Files to modify:

`backend/src/game/actions.ts`

**Dependencies:** TASK-20, TASK-17.

**Exact changes — implement `processSpecialAction(state, player)`:**

1. `rivalId`: the other `PlayerId`.
2. If `player.energy < ENERGY_SPECIAL_COST` → `ENERGY_INSUFFICIENT` (400).
3. `player.energy -= ENERGY_SPECIAL_COST`.
4. Compute Manhattan distance to rival: `|player.x - rival.x| + |player.y - rival.y|`.
5. If distance === 1 (adjacent):
  - Call `applyDamage(state, rivalId, 1, player.id, true)` (**bypass shield = true**).
  - `rival.energy = Math.max(0, rival.energy - ENERGY_SPECIAL_HIT_DRAIN)`.
  - `player.score += SCORE_SPECIAL_HIT`.
  - Call `checkVictory(state)`.
6. If distance > 1:
  - `player.shieldActive = true`.
  - Schedule `setTimeout(SHIELD_DURATION_MS)` that sets `player.shieldActive = false`.
  - Register timeout in `getGameTimers(state.gameId)!.timeouts`.

### Acceptance criteria:

- Rival adjacent: `rival.hp - 1`, `rival.energy - 2` (min 0), `player.score + 2`, `player.energy - 3`. Shield is bypassed.
- Rival not adjacent: `player.shieldActive === true`, no damage.
- After `SHIELD_DURATION_MS`, `player.shieldActive === false`.
- `energy < 3`: error `ENERGY_INSUFFICIENT`.
- `cd backend && npx tsc --noEmit` exits 0.

### Validation:

```
cd backend && npx tsc --noEmit
cd backend && npm run lint
```

**Expected result:** all four actions are implemented. Wave 5 complete.

---

## WAVE 6 — Bomb System

---

### TASK-25 · `scheduleBomb` — register bomb timer

**Objective:** Replace the stub in `bombs.ts` with the real implementation that triggers explosion after `BOMB_TIMER_S` seconds.

#### Files to create/modify:

backend/src/game/bombs.ts

**Dependencies:** TASK-22, TASK-10, TASK-09.

#### Exact changes:

```
export function scheduleBomb(gameId: string, bombId: string): void {
  const timers = getGameTimers(gameId);
  if (!timers) return;
  const t = setTimeout(
    () => triggerExplosion(gameId, bombId, 0),
    BOMB_TIMER_S * 1000
  );
  timers.timeouts.push(t);
}
```

`triggerExplosion` is a stub returning immediately for now (replaced in TASK-26).

**Acceptance criteria:**

- `cd backend && npx tsc --noEmit` exits 0.
- A `setTimeout` is registered in `timers.timeouts` after calling `scheduleBomb`.

**Validation:**

```
cd backend && npx tsc --noEmit
```

**Expected result:** bomb timer is scheduled server-side.

---

## TASK-26 · Explosion radius calculation

**Objective:** Implement the helper that computes all cells affected by an explosion.

**Files to modify:**

```
backend/src/game/bombs.ts
```

**Dependencies:** TASK-25, TASK-07, TASK-09.

**Exact changes:**

```
function computeAffectedCells(
  grid: CellType[][],
  bx: number,
  by: number
): Set<string> {
  const cells = new Set<string>();
  const deltas = [[0,-1],[0,1],[-1,0],[1,0]]; // up,down,left,right
  for (const [dx, dy] of deltas) {
    for (let step = 1; step <= BOMB_RADIUS; step++) {
      const nx = bx + dx * step;
      const ny = by + dy * step;
      if (nx < 0 || nx >= GRID_COLS || ny < 0 || ny >= GRID_ROWS) break;
      if (grid[ny][nx] === 'wall') break; // wall stops wave
      cells.add(`${nx},${ny}`);
      if (grid[ny][nx] === 'destructible') break; // destructible stops further propagation
    }
  }
  return cells;
}
```

Note: the origin cell (`bx`, `by`) is NOT included (bomb is placed there; destroyed destructible at origin would be a separate edge case — the design places bombs only on 'empty' cells).

#### Acceptance criteria:

- Open grid, bomb at (3,3), radius 2: cells (4,3), (5,3), (2,3), (1,3), (3,4), (3,5), (3,2), (3,1) are all included.
- Wall at (4,3): (4,3) is excluded, (5,3) is excluded.
- Destructible at (4,3): (4,3) is included but (5,3) is excluded.
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

`cd backend && npx tsc --noEmit`

**Expected result:** correct explosion geometry respecting wall and destructible blocking.

---

## TASK-27 · `triggerExplosion` — apply all explosion effects

**Objective:** Full implementation of bomb explosion with all effects in the correct order.

#### Files to modify:

`backend/src/game/bombs.ts`

**Dependencies:** TASK-26, TASK-17, TASK-09.

**Exact changes — implement `triggerExplosion(gameId, bombId, depth)`:**

1. Guard: `if (depth > CHAIN_REACTION_MAX_DEPTH) return`.
2. `state = getGame(gameId)`. Guard: if null or `status !== 'playing'` return.
3. `bomb = state.bombs.find(b => b.id === bombId)`. Guard: if null return.
4. `affected = computeAffectedCells(state.grid, bomb.x, bomb.y)`.
5. **Destructibles:** for each cell in `affected` where `grid[y][x] === 'destructible'`:
  - `grid[y][x] = 'empty'`.
  - `if (Math.random() < RESOURCE_SPAWN_CHANCE): add ResourceState { id: uuid, x, y, type: random } to state.resources`.

6. **Player damage** — iterate [ 'player1', 'player2' ] in fixed order. For each: if "px,py" is in `affected` → `applyDamage(state, pid, 1, bomb.playerId, false)`.
7. **Core destruction:** for each core where "cx,cy" in `affected`: remove from `state.cores`, call `scheduleRespawn(gameId, core.id, CORE_RESPAWN_MS)`. No points awarded.
8. **Chain reactions:** collect all bombs (other than `bombId`) whose "bx,by" is in `affected`. Remove them from `state.bombs`, restore their owners' `bombAvailable = true`, then call `triggerExplosion(gameId, otherBomb.id, depth + 1)` for each.
9. Remove `bomb` from `state.bombs`.
10. `state.players[bomb.playerId].bombAvailable = true`.
11. Call `checkVictory(state)`.
12. `setGame(gameId, state)`.

#### Acceptance criteria:

- Bomb explodes after `BOMB_TIMER_S`: confirmed by polling state.
- `state.bombs` is empty after explosion.
- `bombAvailable` restored to `true`.
- Player in radius: `hp - 1`. Player outside: unaffected.
- Player with shield: shield consumed, no hp loss.
- `destructible` in radius: becomes `'empty'`.
- Core in radius: removed, no score change.
- Chain reaction: second bomb in radius of first also explodes (depth 1).
- Max 1 hp damage per player per explosion (guaranteed by `applyDamage` alive-check).
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

```
cd backend && npx tsc --noEmit
cd backend && npm run lint
```

**Expected result:** complete bomb lifecycle from placement to explosion effects.

---

## WAVE 7 — Cores, Resources and Reactor Pulse

---

### TASK-28 · Core respawn — prevent duplicates, respect maximum

**Objective:** Replace the `cores.ts` stub with the full respawn implementation.

**Files to create:**

backend/src/game/cores.ts

**Dependencies:** TASK-10, TASK-07, TASK-09.

**Exact changes:**

```
export function scheduleRespawn(
  gameId: string,
  coreId: string,
  delayMs: number
): void {
  const timers = getGameTimers(gameId);
  if (!timers) return;
  if (timers.respawningCores.has(coreId)) return; // prevent duplicate
  timers.respawningCores.add(coreId);
  const t = setTimeout(() => respawnCore(gameId, coreId), delayMs);
  timers.timeouts.push(t);
}

function respawnCore(gameId: string, coreId: string): void {
  const timers = getGameTimers(gameId);
  if (timers) timers.respawningCores.delete(coreId);

  const state = getGame(gameId);
  if (!state || state.status !== 'playing') return;
  if (state.cores.length >= MAX_CORE_COUNT) return;

  // Find free cell not occupied by players, bombs, other cores, or resources
  const position = findFreeCell(state);
  if (!position) return; // extremely unlikely — grid is full

  state.cores.push({ id: coreId, x: position.x, y: position.y });
  setGame(gameId, state);
}

function findFreeCell(state: GameState): {x:number,y:number} | null {
  // collect all occupied coordinates
  // pick random 'empty' grid cell not in occupied set
}
```

**Acceptance criteria:**

- After capture, core reappears after `CORE_RESPAWN_MS` ms.

- Calling `scheduleRespawn` twice for the same `coreId` schedules only one respawn.
- If `cores.length >= MAX_CORE_COUNT` on respawn, no core is added.
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

```
cd backend && npx tsc --noEmit
cd backend && npm run lint
```

**Expected result:** cores have a complete lifecycle with no duplicates.

---

## TASK-29 · Resource generation helper

**Objective:** Centralise the logic for creating a random `ResourceState` at a given position.

#### Files to modify:

`backend/src/game/cores.ts` OR `backend/src/game/factory.ts`

**Dependencies:** TASK-07, TASK-09.

#### Exact changes:

Export `createRandomResource(x: number, y: number): ResourceState`:

```
export function createRandomResource(x: number, y: number): ResourceState {
  return {
    id: crypto.randomUUID(),
    x, y,
    type: Math.random() < 0.5 ? 'energy_pack' : 'repair_kit'
  };
}
```

Update all callers in `bombs.ts` and `arenaEvent.ts` to use this function.

#### Acceptance criteria:

- Type distribution is approximately 50/50 over many calls.
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

```
cd backend && npx tsc --noEmit
```

**Expected result:** no duplicated resource-creation logic.

---

## TASK-30 · Reactor Pulse — schedule, warn, trigger

**Objective:** Implement `arenaEvent.ts` with the one-time Reactor Pulse event.

**Files to create:**

backend/src/game/arenaEvent.ts

**Dependencies:** TASK-09, TASK-10, TASK-29.

**Exact changes:**

```
export function scheduleReactorPulse(gameId: string): void {
  const timers = getGameTimers(gameId);
  if (!timers) return;

  const delaySec = REACTOR_PULSE_MIN_S +
    Math.floor(Math.random() * (REACTOR_PULSE_MAX_S - REACTOR_PULSE_MIN_S +
1));
  const delayMs = delaySec * 1000;
  const warningMs = delayMs - REACTOR_PULSE_WARNING_S * 1000;

  const t1 = setTimeout(() => activateWarning(gameId), warningMs);
  const t2 = setTimeout(() => triggerReactorPulse(gameId), delayMs);
  timers.timeouts.push(t1, t2);
}

function activateWarning(gameId: string): void {
  const state = getGame(gameId);
  if (!state || state.status !== 'playing') return;
  state.arenaEventActive = true;
  state.arenaEventCountdown = REACTOR_PULSE_WARNING_S;
  setGame(gameId, state);
}

export function triggerReactorPulse(gameId: string): void {
  const state = getGame(gameId);
  if (!state || state.status !== 'playing') return;

  for (let y = 0; y < GRID_ROWS; y++) {
    for (let x = 0; x < GRID_COLS; x++) {
      if (state.grid[y][x] === 'destructible' && (x % 2 === 0 || y % 2 === 0)) {
```

```

    state.grid[y][x] = 'empty';
    if (Math.random() < RESOURCE_SPAWN_CHANCE) {
        state.resources.push(createRandomResource(x, y));
    }
}
}
}
}

state.arenaEventActive = false;
state.arenaEventCountdown = null;
setGame(gameId, state);
// Does NOT call scheduleReactorPulse again — one-time event.
}

```

Wire `scheduleReactorPulse` call in `factory.ts` TASK-19 stub.

#### Acceptance criteria:

- `arenaEventCountdown` goes from `REACTOR_PULSE_WARNING_S` down to 0 over 5 seconds (via tick).
- After `triggerReactorPulse`: all `destructible` at even x or even y become `'empty'`.
- `arenaEventActive === false, arenaEventCountdown === null` after pulse.
- Event does NOT repeat.
- `cd backend && npx tsc --noEmit` exits 0.

#### Validation:

```

cd backend && npx tsc --noEmit
cd backend && npm run lint

```

**Expected result:** Reactor Pulse is a self-contained, one-time event module.

## WAVE 8 — REST API

### TASK-31 · **POST** `/api/game` endpoint

**Objective:** Create a game and return its state.

**Files to create:**

backend/src/routes/game.ts (partial)

**Dependencies:** TASK-14, TASK-19.

**Exact changes:**

```
router.post('/', (req, res) => {  
  const scenario = process.env.NODE_ENV === 'test'  
    ? req.body?.scenario  
    : undefined;  
  const state = createGame(scenario);  
  res.status(200).json({ gameId: state.gameId, state });  
});
```

Register router in `index.ts`: `app.use('/api/game', gameRouter)`.

**Acceptance criteria:**

- `POST /api/game` → 200 with { `gameId`, `state` }.
- `state.status` === `'playing'`.
- `state.players.player1.x` === 1.
- Passing `scenario` in production (`NODE_ENV=production`) is ignored.
- `cd backend && npx tsc --noEmit` exits 0.

**Validation:**

```
cd backend && npm run build  
node backend/dist/index.js &  
curl -s -X POST http://localhost:3000/api/game | jq '{gameId, status: .state.status}'  
kill %1
```

**Expected result:** game creation endpoint works end-to-end.

---

## TASK-32 · `GET /api/game/:gameId` endpoint

**Objective:** Return the current state of the specified game.

**Files to modify:**

backend/src/routes/game.ts

**Dependencies:** TASK-31.

### Exact changes:

```
router.get('/:gameId', (req, res) => {
  const state = getGame(req.params.gameId);
  if (!state) return res.status(404).json({ error: 'Game not found' });
  res.status(200).json(state);
});
```

### Acceptance criteria:

- Valid `gameId` → 200 with full `GameState`.
- Unknown `gameId` → 404 with `{ error: 'Game not found' }`.

### Validation:

```
GAME_ID=$(curl -s -X POST http://localhost:3000/api/game | jq -r .gameId)
curl -s http://localhost:3000/api/game/$GAME_ID | jq .status
curl -o /dev/null -w "%{http_code}" http://localhost:3000/api/game/invalid-id
# must print 404
```

**Expected result:** state polling endpoint works.

---

## TASK-33 · **POST** `/api/game/:gameId/action` endpoint

**Objective:** Receive an action, process it, return new state or error.

### Files to modify:

backend/src/routes/game.ts

**Dependencies:** TASK-20, TASK-21, TASK-22, TASK-23, TASK-24.

### Exact changes:

```
router.post('/:gameId/action', (req, res) => {
  try {
    const state = processAction(req.params.gameId, req.body as ActionRequest);
    res.status(200).json({ success: true, state });
  } catch (err) {
    if (err instanceof GameActionError) {
      return res.status(err.httpStatus).json({
        success: false,
        error: err.code,
        message: err.message
      });
    }
  }
});
```

```

    });
  }
  res.status(500).json({ success: false, error: 'INTERNAL', message: 'Internal error.' });
}
});

```

### Acceptance criteria:

- Valid move → 200 { success:true, state }.
- Invalid move → 400 { success:false, error:'MOVE\_BLOCKED', message:'...' }.
- Finished game action → 409 { success:false, error:'GAME\_NOT\_PLAYING', message:'...' }.
- Unknown gameId → 404.

### Validation:

```

GAME_ID=$(curl -s -X POST http://localhost:3000/api/game | jq -r .gameId)
curl -s -X POST http://localhost:3000/api/game/$GAME_ID/action \
  -H "Content-Type: application/json" \
  -d '{"playerId":"player1","type":"move","payload":{"direction":"right"}}' | jq .success
curl -s -X POST http://localhost:3000/api/game/$GAME_ID/action \
  -H "Content-Type: application/json" \
  -d '{"playerId":"player1","type":"move","payload":{"direction":"left"}}' | jq .error
# left from (1,1) → wall

```

**Expected result:** action endpoint handles all success and error paths.

---

## TASK-34 · Test-only **force-end** endpoint

**Objective:** POST /api/game/:gameId/test/force-end exists only when NODE\_ENV=test. Unreachable in production.

### Files to create:

backend/src/routes/test.ts

### Files to modify:

backend/src/index.ts

**Dependencies:** TASK-16, TASK-32.

### Exact changes:

backend/src/routes/test.ts:

```
router.post('/:gameId/test/force-end', (req, res) => {
  const state = getGame(req.params.gameId);
  if (!state) return res.status(404).json({ error: 'Game not found' });
  state.timeRemaining = 0;
  checkVictory(state);
  setGame(req.params.gameId, state);
  res.status(200).json({ success: true, state });
});
```

backend/src/index.ts — conditional registration:

```
if (process.env.NODE_ENV === 'test') {
  const testRouter = require('./routes/test').default;
  app.use('/api/game', testRouter);
}
```

### Acceptance criteria:

- `NODE_ENV=test`: `POST .../test/force-end` → 200, `state.status === 'finished'`.
- `NODE_ENV=production`: the same URL returns 200 HTML (SPA fallback) or 404 JSON, never a `finished` state.
- The file is not imported at all when `NODE_ENV !== 'test'`.

### Validation:

```
NODE_ENV=test node backend/dist/index.js &
GAME_ID=$(curl -s -X POST http://localhost:3000/api/game | jq -r .gameId)
curl -s -X POST http://localhost:3000/api/game/$GAME_ID/test/force-end | jq .state.status
kill %1
```

```
NODE_ENV=production node backend/dist/index.js &
GAME_ID=$(curl -s -X POST http://localhost:3000/api/game | jq -r .gameId)
STATUS=$(curl -s -X POST http://localhost:3000/api/game/$GAME_ID/test/force-end | jq -r
.state.status 2>/dev/null)
echo $STATUS # must not be "finished"
kill %1
```

**Expected result:** test-only endpoint is invisible in production.

---

## TASK-35 · Express serves compiled frontend

**Objective:** In production, Express serves the Vite build and handles SPA routing.

**Files to modify:**

backend/src/index.ts

**Dependencies:** TASK-03, TASK-05.

**Exact changes:**

After API routers, add:

import path from 'path';

```
app.use(express.static(path.join(__dirname, '../frontend/dist')));
app.get('*', (_req, res) => {
  res.sendFile(path.join(__dirname, '../frontend/dist/index.html'));
});
```

**Acceptance criteria:**

- `npm run build` (root) compiles both packages.
- `node backend/dist/index.js` serves `<h1>Reactor Rush</h1>` at `http://localhost:3000`.
- `/api/game` routes still return JSON (API registered before static handler).

**Validation:**

```
npm run build
node backend/dist/index.js &
curl -s http://localhost:3000 | grep "Reactor Rush"
curl -s http://localhost:3000/api/game/nonexistent | jq .error
kill %1
```

**Expected result:** single URL serves complete application.

---

## WAVE 9 — Frontend API Client and Application State

---

### TASK-36 · Native fetch API client

**Objective:** Implement `frontend/src/api/client.ts` using only the native `fetch` API.

**Files to create:**

`frontend/src/api/client.ts`

**Dependencies:** TASK-08.

**Exact changes:**

```
const BASE = (import.meta.env.VITE_API_BASE as string | undefined) ?? '/api';
```

```
export async function postGame(scenario?: TestScenario):
```

```
  Promise<CreateGameResponse> {  
    const res = await fetch(`${BASE}/game`, {  
      method: 'POST',  
      headers: { 'Content-Type': 'application/json' },  
      body: JSON.stringify(scenario ? { scenario } : {})  
    });  
    if (!res.ok) throw new Error(`HTTP ${res.status}`);  
    return res.json();  
  }  
}
```

```
export async function getGame(gameId: string): Promise<GameState> {
```

```
  const res = await fetch(`${BASE}/game/${gameId}`);  
  if (!res.ok) throw new Error(`HTTP ${res.status}`);  
  return res.json();  
}
```

```
export async function postAction(  
  gameId: string,  
  req: ActionRequest
```

```
): Promise<ActionResponse> {  
  const res = await fetch(`${BASE}/game/${gameId}/action`, {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify(req)  
  });  
  return res.json(); // return body regardless of status; caller checks .success  
}
```

No `axios`, no external HTTP libraries.

**Acceptance criteria:**

- `grep axios frontend/src/api/client.ts` returns empty.

- `cd frontend && npm run typecheck` exits 0.
- All three functions return correctly typed Promises.

#### Validation:

```
cd frontend && npm run typecheck
cd frontend && npm run lint
grep "axios" frontend/src/api/client.ts
```

**Expected result:** complete typed HTTP client using only native `fetch`.

---

## TASK-37 · App-level state machine

**Objective:** `App.tsx` manages three screens (`start`, `game`, `result`) with clean transitions.

#### Files to modify:

```
frontend/src/App.tsx
frontend/src/App.css
```

**Dependencies:** TASK-36, TASK-08.

#### Exact changes:

`App.tsx`:

```
type Screen = 'start' | 'game' | 'result';
```

```
interface AppState {
  screen: Screen;
  gameId: string | null;
  gameState: GameState | null;
}
```

#### State transitions:

- `start` → `game`: `postGame()` succeeds, store `gameId` and `gameState`.
- `game` → `result`: polling detects `state.status === 'finished'`.
- `result` → `start` OR `result` → `game`: "New Game" button calls `postGame()` again.

Render: switch on `screen` to render `<StartScreen>`, `<GameScreen>`, or `<ResultScreen>`. All three components are stubs returning `<div>` for now (implemented in Wave 11).

```
App.css:html, body { margin:0; height:100%; } #root { height:100vh;
}.
```

#### Acceptance criteria:

- `cd frontend && npm run typecheck` exits 0.
- `cd frontend && npm run build` exits 0.
- No `react-router`, no `redux`.

#### Validation:

```
cd frontend && npm run typecheck
cd frontend && npm run build
grep -E "react-router|redux" frontend/src/App.tsx
```

**Expected result:** application lifecycle state machine in place.

---

## WAVE 10 — Keyboard Controls

---

### TASK-38 · `useKeyboard` hook

**Objective:** Capture player input, send action to backend, return the response for error display.

#### Files to create:

frontend/src/hooks/useKeyboard.ts

**Dependencies:** TASK-36, TASK-08.

#### Exact changes:

```
const KEY_MAP: Record<string, ActionRequest> = {
  'w': { playerId:'player1', type:'move', payload:{direction:'up'} },
  'a': { playerId:'player1', type:'move', payload:{direction:'left'} },
  's': { playerId:'player1', type:'move', payload:{direction:'down'} },
  'd': { playerId:'player1', type:'move', payload:{direction:'right'} },
  'f': { playerId:'player1', type:'place_bomb' },
  'g': { playerId:'player1', type:'capture_core' },
  'e': { playerId:'player1', type:'special_action' },
  'ArrowUp': { playerId:'player2', type:'move', payload:{direction:'up'} },
  'ArrowLeft': { playerId:'player2', type:'move', payload:{direction:'left'} },
  'ArrowDown': { playerId:'player2', type:'move', payload:{direction:'down'} },
```

```

'ArrowRight': { playerId:'player2', type:'move', payload:{direction:'right'} },
'l': { playerId:'player2', type:'place_bomb' },
'k': { playerId:'player2', type:'capture_core' },
'o': { playerId:'player2', type:'special_action' },
};

export function useKeyboard(
  gameId: string,
  onResult: (res: ActionResponse) => void
): void {
  useEffect(() => {
    const handler = (e: KeyboardEvent) => {
      if (e.repeat) return; // ignore key-repeat
      const action = KEY_MAP[e.key];
      if (!action) return;
      e.preventDefault();
      postAction(gameId, action).then(onResult).catch(console.error);
    };
    window.addEventListener('keydown', handler);
    return () => window.removeEventListener('keydown', handler);
  }, [gameId, onResult]);
}

```

No debounce. No continuous movement. One keydown = one action.

### Acceptance criteria:

- `event.repeat === true` → no request sent.
- Unrecognised key → no request sent.
- `W` maps to `player1 move up`.
- `ArrowUp` maps to `player2 move up`.
- `cd frontend && npm run typecheck` exits 0.

### Validation:

```

cd frontend && npm run typecheck
cd frontend && npm run lint
grep "debounce\|setTimeout" frontend/src/hooks/useKeyboard.ts
# must return empty

```

**Expected result:** discrete, correct keyboard-to-action mapping with no flooding.

---

## TASK-39 · `usePolling` hook

**Objective:** Poll `GET /api/game/:gameId` every 500 ms and forward state updates.

**Files to create:**

frontend/src/hooks/usePolling.ts

**Dependencies:** TASK-36, TASK-08.

**Exact changes:**

```
export function usePolling(
  gameId: string,
  onState: (state: GameState) => void,
  intervalMs: number = 500
): void {
  useEffect(() => {
    const id = setInterval(async () => {
      try {
        const state = await getGame(gameId);
        onState(state);
        if (state.status === 'finished') clearInterval(id);
      } catch { /* network blip — continue polling */ }
    }, intervalMs);
    return () => clearInterval(id);
  }, [gameId, onState, intervalMs]);
}
```

**Acceptance criteria:**

- Calls `getGame` at most once per `intervalMs`.
- Stops polling when `status === 'finished'`.
- Cleanup on unmount cancels the interval.
- `cd frontend && npm run typecheck` exits 0.

**Validation:**

`cd frontend && npm run typecheck`

**Expected result:** self-managing polling hook that stops on game completion.

---

## WAVE 11 — Arena and UI

---

## TASK-40 · Arena grid component

**Objective:** Render the 15×11 grid with correct cell types and overlay elements.

**Files to create:**

frontend/src/components/Arena.tsx  
frontend/src/components/Arena.css

**Dependencies:** TASK-08.

**Exact changes:**

Arena.tsx:

- Accepts full `GameState` as prop.
- Renders a CSS Grid of 15 columns × 11 rows.
- For each `(x, y)` pair, determine and apply: base cell class (`cell--wall`, `cell--destructible`, `cell--empty`).
- Overlay in priority order (highest visible first): player, bomb, core, resource.
- Each cell element: `data-cell-x={x}` and `data-cell-y={y}` attributes (used by E2E tests).
- Wrapping element: `data-testid="arena"`.

Arena.css:

- `.arena { display: grid; grid-template-columns: repeat(15, 1fr); width: 100%; }`
- `.cell { aspect-ratio: 1; position: relative; }`
- `.cell--wall { background: #444; }`
- `.cell--destructible { background: #8B6914; }`
- `.cell--empty { background: #c8d8a0; }`

**Acceptance criteria:**

- Given a `GameState`, the component renders exactly 165 cells (`15 * 11`).
- A cell at a `wall` position has class `cell--wall`.
- Player at `(1,1)` renders inside the cell at column 2, row 2 of the grid.
- `cd frontend && npm run typecheck` exits 0.

**Validation:**

`cd frontend && npm run typecheck`  
`cd frontend && npm run build`

**Expected result:** playable grid rendered purely with React + CSS, no canvas or game engine.

---

## TASK-41 · Player rendering with shield indicator

**Objective:** Each player is rendered in their current grid cell with a visual shield indicator when active.

### Files to create:

frontend/src/components/PlayerSprite.tsx  
frontend/src/components/PlayerSprite.css

**Dependencies:** TASK-40.

### Exact changes:

PlayerSprite.tsx:

- Props: `player: PlayerState`.
- Renders a `<div>` with class `player--p1` or `player--p2`.
- If `shieldActive`: adds class `player--shielded`.
- If `!alive`: adds class `player--dead`.
- `data-testid={player-${player.id}}`.

PlayerSprite.css:

- `.player--p1 { background: #3b82f6; border-radius: 50%; width: 80%; height: 80%; margin: 10%; }`
- `.player--p2 { background: #f97316; ... }`
- `.player--shielded { box-shadow: 0 0 6px 3px white; animation: pulse 0.5s infinite alternate; }`
- `.player--dead { opacity: 0.3; }`
- `@keyframes pulse` for subtle shield animation.

Arena `renderCell` overlays `<PlayerSprite>` when `player.x === x && player.y === y`.

### Acceptance criteria:

- P1 renders blue, P2 renders orange.
- Shield class present when `shieldActive === true`.
- `data-testid="player-player1"` and `data-testid="player-player2"` present in DOM.

- `cd frontend && npm run typecheck` exits 0.

**Validation:**

`cd frontend && npm run typecheck`

**Expected result:** players are visually distinct and shield state is visible.

---

## TASK-42 · Bomb rendering with countdown visual

**Objective:** Bombs render in their cell with a colour/size change at each second of countdown.

**Files to create:**

frontend/src/components/BombSprite.tsx  
frontend/src/components/BombSprite.css

**Dependencies:** TASK-40.

**Exact changes:**

**BombSprite.tsx:**

- Props: `bomb: BombState`.
- Returns a `<div>` with dynamic class based on `timerRemaining`:
  - 3: `bomb--safe` (grey).
  - 2: `bomb--warn` (yellow).
  - 1 or 0: `bomb--danger` (red, slightly larger).

**BombSprite.css:** define the three classes with colour and optionally `transform: scale()`.

**Acceptance criteria:**

- `timerRemaining === 1` → class `bomb--danger` present.
- `timerRemaining === 3` → class `bomb--safe` present.
- `cd frontend && npm run typecheck` exits 0.

**Validation:**

`cd frontend && npm run typecheck`

**Expected result:** countdown is visually apparent without opening the console.

---

## TASK-43 · Core and resource rendering

**Objective:** Cores and resources render with distinct visual identities.

**Files to create:**

frontend/src/components/CoreSprite.tsx  
frontend/src/components/ResourceSprite.tsx  
frontend/src/components/Sprites.css

**Dependencies:** TASK-40.

**Exact changes:**

`CoreSprite.tsx`: renders a diamond/star shape using CSS (`rotate: 45deg` box) in bright yellow. `data-testid="core"`.

`ResourceSprite.tsx`: props `type: ResourceType.energy_pack` → cyan circle. `repair_kit` → green cross shape. `data-testid="resource"`.

No SVG from external sources, no image files — pure CSS shapes.

**Acceptance criteria:**

- Cores and resources render at correct grid positions.
- The two resource types look visually different.
- `cd frontend && npm run typecheck` exits 0.

**Validation:**

`cd frontend && npm run typecheck`  
`cd frontend && npm run build`

**Expected result:** all map elements are distinguishable without documentation.

---

## TASK-44 · PlayerHUD component

**Objective:** Each HUD shows hp, energy, resources, score, bomb availability, and shield — plus error messages.

**Files to create:**

frontend/src/components/PlayerHUD.tsx  
frontend/src/components/PlayerHUD.css

**Dependencies:** TASK-08.

**Exact changes:**

PlayerHUD.tsx:

- Props: `player: PlayerState`, `label: string`, `errorMessage: string | null`.
- HP: render `MAX_HP` heart icons, filled for current hp, empty for lost hp.
- Energy: `<div>` bar with `width: ${({energy/MAX_ENERGY})*100}%`.
- Resources: plain counter number.
- Score: plain counter number.
- Bomb: "💣 ✓" or "💣 ✗".
- Shield: highlighted indicator when `shieldActive`.
- Error: `<div data-testid="error-${player.id}" class="hud__error">`  
— visible only when `errorMessage !== null`.

PlayerHUD.css: layout, colours, sizes. No external library.

**Acceptance criteria:**

- `hp === 2`: 2 full hearts, 1 empty heart.
- `bombAvailable === false`: bomb indicator shows ✗.
- `errorMessage !== null`: error div is visible in the DOM with the message text.
- `data-testid="error-player1"` and `data-testid="error-player2"` present.
- `cd frontend && npm run typecheck` exits 0.

**Validation:**

`cd frontend && npm run typecheck`

**Expected result:** all player stats and errors are visible in the HUD.

---

## TASK-45 · Time display and Reactor Pulse warning

**Objective:** Show time remaining centred at the top; show a warning when `arenaEventCountdown !== null`.

**Files to create:**

frontend/src/components/TimeDisplay.tsx  
frontend/src/components/TimeDisplay.css

**Dependencies:** TASK-08.

**Exact changes:**

`TimeDisplay.tsx`:

- Props: `timeRemaining: number`, `arenaEventActive: boolean`, `arenaEventCountdown: number | null`.
- Format time as `MM:SS`.
- If `arenaEventCountdown !== null`: render `<div data-testid="reactor-warning">PULSO EN {arenaEventCountdown}s</div>` with a prominent warning style.
- `data-testid="time-remaining"` on the time element.

**Acceptance criteria:**

- `timeRemaining === 75` → shows `01:15`.
- `arenaEventCountdown === 3` → warning text visible.
- `arenaEventCountdown === null` → warning element not rendered.
- `cd frontend && npm run typecheck` exits 0.

**Validation:**

`cd frontend && npm run typecheck`

**Expected result:** time and event warning are always visible during play.

---

## TASK-46 · StartScreen component

**Objective:** Full start screen with game title, objective summary, controls table, and start button.

**Files to create:**

`frontend/src/components/StartScreen.tsx`  
`frontend/src/components/StartScreen.css`

**Dependencies:** TASK-36, TASK-08.

**Exact changes:**

`StartScreen.tsx`:

- Props: `onStart: (gameId: string, state: GameState) => void`.
- On button click: call `postGame()` → on success call `onStart(gameId, state)`.
- Display: game name, one-sentence objective, two-column controls table (P1 / P2), start button.

#### Acceptance criteria:

- `data-testid="start-button"` present on the button.
- Clicking button calls `POST /api/game` (verifiable in Playwright network intercept).
- `cd frontend && npm run typecheck` exits 0.

#### Validation:

```
cd frontend && npm run typecheck
cd frontend && npm run build
```

**Expected result:** visually complete start screen.

---

## TASK-47 · ResultScreen component

**Objective:** Show winner / draw, final scores, and new-game button.

#### Files to create:

```
frontend/src/components/ResultScreen.tsx
frontend/src/components/ResultScreen.css
```

**Dependencies:** TASK-08.

#### Exact changes:

`ResultScreen.tsx`:

- Props: `state: GameState, onNewGame: () => void`.
- `data-testid="result-screen"` on root element.
- Display result text based on `state.result`.
- Display `state.players.player1.score` and `state.players.player2.score`.
- "Nueva partida" button calls `onNewGame`.

#### Acceptance criteria:

- `result === 'draw'` → text contains "EMPATE" (case-insensitive).
- `result === 'player1_wins'` → text contains "Jugador 1".

- `data-testid="result-screen"` present.
- `cd frontend && npm run typecheck` exits 0.

**Validation:**

`cd frontend && npm run typecheck`

**Expected result:** result screen clearly communicates game outcome.

---

## TASK-48 · GameScreen component

**Objective:** Compose Arena, HUDs, TimeDisplay, and error messages into the full game layout.

**Files to create:**

frontend/src/components/GameScreen.tsx  
 frontend/src/components/GameScreen.css

**Dependencies:** TASK-39, TASK-38, TASK-40, TASK-44, TASK-45.

**Exact changes:**

`GameScreen.tsx`:

- Props: `gameId: string, initialState: GameState, onFinish: (state: GameState) => void`.
- Local state: `gameState, errors: { player1: string | null; player2: string | null }`.
- `usePolling(gameId, handleState)`:
  - `handleState` updates `gameState`; if `status === 'finished'` calls `onFinish`.
- `useKeyboard(gameId, handleResult)`:
  - `handleResult`: if `!res.success`, set error for the relevant player; `setTimeout(2000)` clears it.
- Layout (CSS Grid, 3 columns: 200px | 1fr | 200px, height 100vh):
  - Left: `<PlayerHUD player={p1} errorMessage={errors.player1} />`.
  - Centre: `<TimeDisplay>` above `<Arena>`.
  - Right: `<PlayerHUD player={p2} errorMessage={errors.player2} />`.

`GameScreen.css`: three-column grid, fills viewport.

**Acceptance criteria:**

- Error message appears in correct HUD within one event loop tick after failed action.
- Error clears after 2 seconds.
- `status === 'finished'` triggers `onFinish` on next poll.
- `cd frontend && npm run typecheck` exits 0.

**Validation:**

```
cd frontend && npm run typecheck
```

```
cd frontend && npm run build
```

**Expected result:** all UI elements composed into a playable layout.

---

## WAVE 12 — Frontend/Backend Integration

---

### TASK-49 · Wire App.tsx with real screen components

**Objective:** Replace screen stubs in `App.tsx` with real components. Full game lifecycle works end-to-end.

**Files to modify:**

```
frontend/src/App.tsx
```

**Dependencies:** TASK-37, TASK-46, TASK-47, TASK-48.

**Exact changes:**

- Import `StartScreen`, `GameScreen`, `ResultScreen`.
- `screen === 'start':` render `<StartScreen onStart={handleStart} />`.
- `screen === 'game':` render `<GameScreen gameId={gameId!} initialState={gameState!} onFinish={handleFinish} />`.
- `screen === 'result':` render `<ResultScreen state={gameState!} onNewGame={handleNewGame} />`.
- `handleStart(gameId, state):` set `screen='game'`, `gameId`, `gameState`.
- `handleFinish(state):` set `screen='result'`, update `gameState`.
- `handleNewGame():` call `postGame()`, transition to `'game'` on success.

**Acceptance criteria:**

- Clicking "Iniciar partida" shows the arena.

- Pressing WASD moves P1 (visible after next poll  $\leq 500$  ms).
- Pressing arrow keys moves P2.
- Game over → result screen appears automatically.
- "Nueva partida" → start screen or new game directly.
- `npm run build` exits 0.

#### Validation:

```
npm run build
node backend/dist/index.js &
# manual browser test at http://localhost:3000
kill %1
```

**Expected result:** complete end-to-end game is playable in the browser.

---

## TASK-50 · Integration verification — timer cleanup

**Objective:** Verify no timer leaks occur when a game is replaced. Manual curl test.

**Files to modify:** none (verification only; create fix if needed).

**Dependencies:** TASK-15, TASK-49.

#### Exact changes:

Run verification script:

```
npm run build
NODE_ENV=development node backend/dist/index.js &
GAME1=$(curl -s -X POST http://localhost:3000/api/game | jq -r .gameId)
sleep 5
T1=$(curl -s http://localhost:3000/api/game/$GAME1 | jq .timeRemaining)
echo "Game1 timeRemaining after 5s: $T1" # should be ~115

GAME2=$(curl -s -X POST http://localhost:3000/api/game | jq -r .gameId)
sleep 5
STATUS=$(curl -o /dev/null -w "%{http_code}" http://localhost:3000/api/game/$GAME1)
echo "Game1 after replace: $STATUS" # must be 404

T2=$(curl -s http://localhost:3000/api/game/$GAME2 | jq .timeRemaining)
echo "Game2 timeRemaining: $T2" # should be ~115
kill %1
```

#### Acceptance criteria:

- Game1 returns 404 after Game2 is created.
- Game2 has `timeRemaining ≈ 115` after 5 seconds (independent clock).
- No console errors about undefined game timers.

**Validation:** script above.

**Expected result:** confirmed no timer leaks on game replacement.

---

## WAVE 13 — E2E Tests

---

### TASK-51 · Playwright setup and configuration

**Objective:** Playwright installed, configured for both local/CI and production modes.

**Files to create:**

tests/playwright.config.ts

**Files to modify:**

package.json (root)

**Dependencies:** TASK-49.

**Exact changes:**

Install: `npm install -D @playwright/test wait-on` at root.

`tests/playwright.config.ts:`

```
import { defineConfig } from '@playwright/test';
```

```
const isProd = process.env.TEST_ENV === 'production';
```

```
export default defineConfig({  
  testDir: './tests/e2e',  
  use: {  
    baseURL: isProd  
      ? (process.env.PRODUCTION_URL ?? 'http://localhost:3000')  
      : 'http://localhost:3000',  
    headless: true,  
  },  
  reporter: [['html', { open: 'never' }], ['list']],  
});
```

```
});
```

Add BFS helper shared by production E2E tests:

tests/e2e/helpers/bfs.ts

bfs.ts exports `findPath(grid: CellType[][], from: {x,y}, to: {x,y}): Direction[] | null`. Standard BFS returning sequence of directions. Used only in test files, not in production code.

Update root `package.json` scripts:

- `"test:e2e": "playwright test --config=tests/playwright.config.ts"`.
- `"test:e2e:headed": "playwright test --config=tests/playwright.config.ts --headed"`.

#### Acceptance criteria:

- `npx playwright install chromium` exits 0.
- `npm run test:e2e` launches Playwright (may fail — no tests yet).
- `bfs.ts` compiles without errors.

#### Validation:

```
npx playwright install chromium
cd tests && npx tsc --noEmit
```

**Expected result:** Playwright ready for test implementation.

---

## TASK-52 · T-01 — Game start

**Objective:** Verify start screen renders and clicking the button shows the arena.

#### Files to create:

tests/e2e/start.spec.ts

**Dependencies:** TASK-51.

#### Exact changes:

```
test('T-01: game start', async ({ page }) => {
```

```
await page.goto('/');
await expect(page.getByText('Reactor Rush')).toBeVisible();
await expect(page.getByTestId('start-button')).toBeVisible();

await page.getByTestId('start-button').click();

await expect(page.getByTestId('arena')).toBeVisible({ timeout: 3000 });
await expect(page.getByTestId('player-player1')).toBeVisible();
await expect(page.getByTestId('player-player2')).toBeVisible();
});
```

### Acceptance criteria:

- Test passes in headless mode.
- Test passes in headed mode (`--headed`).
- Works against both local/CI and production URL.

### Validation:

```
npm run build && NODE_ENV=test node backend/dist/index.js &
sleep 2
npm run test:e2e -- tests/e2e/start.spec.ts
kill %1
```

**Expected result:** T-01 green.

---

## TASK-53 · T-02 — Player movement

**Objective:** Verify that sending `move right` for P1 updates position in the returned state.

### Files to create:

tests/e2e/movement.spec.ts

**Dependencies:** TASK-51.

### Exact changes:

```
test('T-02: player movement', async ({ request }) => {
  const create = await request.post('/api/game');
  const { gameId, state } = await create.json();
  const initialX = state.players.player1.x; // 1

  const moveRes = await request.post(`/api/game/${gameId}/action`, {
```

```

    data: { playerId: 'player1', type: 'move', payload: { direction: 'right' } }
  });
  expect(moveRes.ok()).toBe(true);
  const { state: newState } = await moveRes.json();
  expect(newState.players.player1.x).toBe(initialX + 1);
  expect(newState.players.player1.y).toBe(1);
});

```

#### Acceptance criteria:

- `player1.x` increases by 1.
- Works in local/CI and against production URL.

#### Validation:

npm run test:e2e -- tests/e2e/movement.spec.ts

**Expected result:** T-02 green.

## TASK-54 · T-03 — Polling / state synchronisation

**Objective:** Verify the frontend polls, `GET /api/game/:id` returns a valid `GameState`, and the displayed time decrements.

#### Files to create:

tests/e2e/polling.spec.ts

**Dependencies:** TASK-51.

#### Exact changes:

```

test('T-03: polling and state structure', async ({ page, request }) => {
  // API structure
  const res = await request.post('/api/game');
  const { state } = await res.json();
  for (const key of
['gameId', 'status', 'timeRemaining', 'players', 'bombs', 'cores', 'resources', 'grid']) {
    expect(state).toHaveProperty(key);
  }

  // UI: time decrements
  await page.goto('/');
  await page.getByTestId('start-button').click();

```

```
await expect(page.getByTestId('arena')).toBeVisible();

const t1 = Number(await page.getByTestId('time-remaining').textContent());
await page.waitForTimeout(1600);
const t2 = Number(await page.getByTestId('time-remaining').textContent());
expect(t2).toBeLessThan(t1);
});
```

#### Acceptance criteria:

- All required **GameState** keys present.
- Displayed time decrements within 1.6 seconds.

#### Validation:

```
npm run test:e2e -- tests/e2e/polling.spec.ts
```

**Expected result:** T-03 green.

---

## TASK-55 · T-04 — Core capture (local/CI mode)

**Objective:** Deterministic core capture using **TestScenario**. Verify score +5 and energy +2.

#### Files to create:

```
tests/e2e/capture.spec.ts
```

**Dependencies:** TASK-51, TASK-34.

#### Exact changes — local/CI branch:

```
test('T-04 (local): core capture deterministic', async ({ request }) => {
  if (process.env.TEST_ENV === 'production') test.skip();

  const res = await request.post('/api/game', {
    data: {
      scenario: {
        destructibles: [],
        cores: [{ x: 3, y: 1 }],
        resources: []
      }
    }
  });
  const { gameId } = await res.json();
```

```

// Move P1 from (1,1) to (3,1): right × 2
for (let i = 0; i < 2; i++) {
  await request.post(`/api/game/${gameId}/action`, {
    data: { playerId: 'player1', type: 'move', payload: { direction: 'right' } }
  });
}

const captureRes = await request.post(`/api/game/${gameId}/action`, {
  data: { playerId: 'player1', type: 'capture_core' }
});
expect(captureRes.ok()).toBe(true);
const { state } = await captureRes.json();

expect(state.players.player1.score).toBe(5);    // SCORE_CAPTURE_CORE
expect(state.players.player1.energy).toBe(7);   // 5 + ENERGY_CAPTURE_CORE
expect(state.cores).toHaveLength(0);           // removed, respawn pending
});

```

#### Acceptance criteria:

- `score === 5, energy === 7, cores.length === 0.`
- Test passes in headless mode.

#### Validation:

`NODE_ENV=test npm run test:e2e -- tests/e2e/capture.spec.ts`

**Expected result:** T-04 local/CI green.

## TASK-56 · T-04 — Core capture (production mode)

**Objective:** Production-safe core capture using real game state and BFS pathfinding in the test.

#### Files to modify:

`tests/e2e/capture.spec.ts`

**Dependencies:** TASK-55.

#### Exact changes — production branch (added to same file):

```
test('T-04 (prod): core capture adaptive', async ({ request }) => {
```

```

if (process.env.TEST_ENV !== 'production') test.skip();

const res = await request.post('/api/game');
const { gameId, state: initial } = await res.json();

// Find nearest reachable core from P1 spawn
let target = initial.cores[0];
// If multiple cores, pick closest by Manhattan distance
for (const c of initial.cores) {
  if (Math.abs(c.x-1)+Math.abs(c.y-1) < Math.abs(target.x-1)+Math.abs(target.y-1))
    target = c;
}

const path = findPath(initial.grid, { x: 1, y: 1 }, target);
expect(path).not.toBeNull();

let currentState = initial;
for (const direction of path!) {
  const r = await request.post(`/api/game/${gameId}/action`, {
    data: { playerId: 'player1', type: 'move', payload: { direction } }
  });
  const body = await r.json();
  if (body.state) currentState = body.state;
}

const captureRes = await request.post(`/api/game/${gameId}/action`, {
  data: { playerId: 'player1', type: 'capture_core' }
});
expect(captureRes.ok()).toBe(true);
const { state: final } = await captureRes.json();

expect(final.players.player1.score).toBeGreaterThanOrEqual(5);
});

```

### Acceptance criteria:

- No use of `scenario`, no use of `force-end`.
- Test computes path from real grid state.
- Passes against real production URL.
- `findPath` is imported from `tests/e2e/helpers/bfs.ts`.

### Validation:

```

TEST_ENV=production PRODUCTION_URL=https://reactor-rush.railway.app \
npm run test:e2e -- tests/e2e/capture.spec.ts

```

**Expected result:** T-04 production green.

---

## TASK-57 · T-05 — Invalid special action

**Objective:** Verify that a `special_action` with insufficient energy returns 400 and shows an error in the UI.

**Files to create:**

tests/e2e/invalid-action.spec.ts

**Dependencies:** TASK-51.

**Exact changes:**

```
test('T-05: invalid special action', async ({ page, request }) => {
  // API level: drain energy below 3, then attempt special_action
  const scenario = process.env.TEST_ENV !== 'production'
    ? { scenario: { destructibles: [], cores: [], resources: [] } }
    : {};

  const res = await request.post('/api/game', { data: scenario });
  const { gameId } = await res.json();

  // Use special_action once (costs 3 energy): player now has 2 energy
  await request.post(`/api/game/${gameId}/action`, {
    data: { playerId: 'player1', type: 'special_action' }
  });

  // Now energy = 2 → next special_action must fail
  const errRes = await request.post(`/api/game/${gameId}/action`, {
    data: { playerId: 'player1', type: 'special_action' }
  });
  expect(errRes.status()).toBe(400);
  const body = await errRes.json();
  expect(body.error).toBe('ENERGY_INSUFFICIENT');
  expect(body.success).toBe(false);

  // UI: start a fresh game, trigger the error via keyboard, check HUD message
  await page.goto('/');
  await page.getByTestId('start-button').click();
  await expect(page.getByTestId('arena')).toBeVisible();

  // Drain energy via API for the game the UI is watching — use same gameId trick
  // (simpler: just navigate and press E twice in quick succession)
  await page.keyboard.press('e');
```

```
await page.waitForTimeout(200);
await page.keyboard.press('e');
```

```
await expect(page.getByTestId('error-player1')).toBeVisible({ timeout: 3000 });
});
```

#### Acceptance criteria:

- API returns 400 `ENERGY_INSUFFICIENT`.
- `error-player1` visible in UI.
- Works in local/CI; production branch falls back to natural energy drain.

#### Validation:

`NODE_ENV=test npm run test:e2e -- tests/e2e/invalid-action.spec.ts`

**Expected result:** T-05 green.

---

## TASK-58 · T-06 — Game finalization (local/CI mode)

**Objective:** Use `force-end` to trigger game completion, verify result screen appears.

#### Files to create:

`tests/e2e/finish.spec.ts`

**Dependencies:** TASK-34, TASK-51.

#### Exact changes — local/CI branch:

```
test('T-06 (local): game finalization via force-end', async ({ page, request }) => {
  if (process.env.TEST_ENV === 'production') test.skip();
```

```
  const res = await request.post('/api/game');
  const { gameId } = await res.json();
```

```
  await page.goto('/');
  await page.getByTestId('start-button').click();
  await expect(page.getByTestId('arena')).toBeVisible();
```

```
  await request.post(`/api/game/${gameId}/test/force-end`);
```

```
  // Polling detects finished state within 2 intervals
  await expect(page.getByTestId('result-screen')).toBeVisible({ timeout: 2000 });
```

```
await expect(page.getByTestId('result-screen')).toContainText(/jugador|empate/i);
await expect(page.getByRole('button', { name: /nueva partida/i })).toBeVisible();
});
```

#### Acceptance criteria:

- Result screen appears within 2 seconds of **force-end**.
- Result text matches expected pattern.
- "Nueva partida" button is present.

#### Validation:

NODE\_ENV=test npm run test:e2e -- tests/e2e/finish.spec.ts

**Expected result:** T-06 local/CI green.

---

### TASK-59 · T-06 — Game finalization (production mode)

**Objective:** Reach 25 points through real gameplay. No **force-end**.

#### Files to modify:

tests/e2e/finish.spec.ts

**Dependencies:** TASK-56, TASK-58.

#### Exact changes — production branch:

```
test('T-06 (prod): game finalization via real gameplay', async ({ page, request }) => {
  if (process.env.TEST_ENV !== 'production') test.skip();
```

```
  const res = await request.post('/api/game');
  const { gameId, state: initial } = await res.json();
```

```
  await page.goto('/');
  await page.getByTestId('start-button').click();
  await expect(page.getByTestId('arena')).toBeVisible();
```

```
  let current = initial;
  // Capture cores until score >= VICTORY_SCORE (25 points, 5 captures)
  while (current.players.player1.score < 25 && current.status === 'playing') {
    // Re-poll if no cores available
    if (current.cores.length === 0) {
      await page.waitForTimeout(1000);
```

```

    current = await (await request.get(`/api/game/${gameId}`).json());
    continue;
  }

  const target = current.cores[0];
  const path = findPath(current.grid, current.players.player1, target);
  if (!path) {
    await page.waitForTimeout(1000);
    current = await (await request.get(`/api/game/${gameId}`).json());
    continue;
  }

  for (const direction of path) {
    const r = await request.post(`/api/game/${gameId}/action`, {
      data: { playerId: 'player1', type: 'move', payload: { direction } }
    });
    const b = await r.json();
    if (b.state) current = b.state;
  }

  const captureRes = await request.post(`/api/game/${gameId}/action`, {
    data: { playerId: 'player1', type: 'capture_core' }
  });
  const b = await captureRes.json();
  if (b.state) current = b.state;
}

// Result screen must appear
await expect(page.getByTestId('result-screen')).toBeVisible({ timeout: 5000 });
});

```

### Acceptance criteria:

- No test-only endpoints used.
- Result screen appears after real gameplay reaches 25 points.
- Timeout set to 120 000 ms (full game duration) in `playwright.config.ts` for this test.

### Validation:

```

TEST_ENV=production PRODUCTION_URL=https://reactor-rush.railway.app \
npm run test:e2e -- tests/e2e/finish.spec.ts --timeout 120000

```

**Expected result:** T-06 production green.

---

## WAVE 14 — GitHub Actions

---

### TASK-60 · Lint workflow

**Objective:** ESLint runs on every push and pull request for both packages.

**Files to create:**

.github/workflows/lint.yml

**Dependencies:** TASK-06.

**Exact changes:**

```
name: Lint
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run lint --workspace=backend
      - run: npm run lint --workspace=frontend
```

**Acceptance criteria:**

- Workflow appears in GitHub Actions tab after push.
- Fails if a lint error is introduced.
- Passes on clean code.

**Validation:** push to GitHub, observe Actions tab.

**Expected result:** automated lint gate on every commit.

---

### TASK-61 · E2E workflow

**Objective:** Build project, start server, run all 6 Playwright tests headless on every push/PR.

**Files to create:**

.github/workflows/e2e.yml

**Dependencies:** TASK-59, TASK-51.

**Exact changes:**

```
name: E2E Tests
on: [push, pull_request]
jobs:
  e2e:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run build
      - run: npx playwright install --with-deps chromium
      - name: Start server
        run: NODE_ENV=test node backend/dist/index.js &
      - name: Wait for server
        run: npx wait-on http://localhost:3000/health --timeout 15000
      - name: Run E2E
        run: npm run test:e2e
    env:
      NODE_ENV: test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
```

**Acceptance criteria:**

- All 6 tests pass in CI.
- Playwright HTML report uploaded as artifact on failure.
- Workflow fails if any test fails.

**Validation:** push to GitHub, observe Actions tab.

**Expected result:** automated E2E gate with failure artifacts.

---

## TASK-62 · Deploy workflow (Railway)

**Objective:** Automatic deployment to Railway on push to `main`.

**Files to create:**

.github/workflows/deploy.yml  
railway.toml

**Dependencies:** TASK-35.

**Platform decision: Railway** (chosen for zero-config Express + static serving, free tier, GitHub Actions integration via CLI token, environment variable management).

**Exact changes:**

railway.toml:

```
[build]
builder = "NIXPACKS"
buildCommand = "npm run build"
```

```
[deploy]
startCommand = "npm start"
healthcheckPath = "/health"
```

.github/workflows/deploy.yml:

```
name: Deploy
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run build
      - run: npm install -g @railway/cli
      - run: railway up --service reactor-rush --detach
    env:
      RAILWAY_TOKEN: ${ secrets.RAILWAY_TOKEN }
```

Required GitHub secrets: **RAILWAY\_TOKEN**.

Railway environment variables (set in Railway dashboard):

- `NODE_ENV=production`
- `PORT=3000` (Railway injects `PORT` automatically)

**Acceptance criteria:**

- Push to `main` triggers deployment.
- `https://<app>.railway.app/health` returns `{"status":"ok"}` after deployment.
- `https://<app>.railway.app` serves the React app.

**Validation:**

`curl https://reactor-rush.railway.app/health`

**Expected result:** fully automated deployment pipeline to Railway.

---

## WAVE 15 — Deployment

---

### TASK-63 · Railway service configuration

**Objective:** Configure the Railway service with correct build/start commands, environment variables, and health check.

**Files to create/modify:**

`railway.toml` (created in TASK-62)

**Dependencies:** TASK-62, TASK-35.

**Exact changes:**

- Verify `railway.toml` build command is `npm run build` (runs both backend and frontend).
- Verify start command is `npm start` which runs `node backend/dist/index.js`.
- Set Railway dashboard variables: `NODE_ENV=production`.
- Health check path: `/health`.

**Acceptance criteria:**

- Manual deployment via `railway up` from local machine succeeds.
- Public URL responds at `/health` with `{"status":"ok"}`.
- Public URL serves the React SPA at `/`.

- `/api/game` responds to POST.

**Validation:**

```
railway up
curl https://<app>.railway.app/health
curl -X POST https://<app>.railway.app/api/game | jq .gameId
```

**Expected result:** production deployment verified working.

---

## TASK-64 · Production E2E smoke test

**Objective:** Run T-04 and T-06 production variants against the live Railway URL.

**Dependencies:** TASK-56, TASK-59, TASK-63.

**Exact changes:** none — run existing tests with environment variables.

**Validation:**

```
TEST_ENV=production \
PRODUCTION_URL=https://reactor-rush.railway.app \
npm run test:e2e -- --timeout 120000
```

**Acceptance criteria:**

- T-01 through T-06 all pass against production URL.
- Production variants of T-04 and T-06 do not use `scenario` or `force-end`.

**Expected result:** full E2E suite verified against live deployment.

---

## WAVE 16 — Documentation

---

### TASK-65 · `docs/introduccion.md`

**Files to create:** `docs/introduccion.md`

**Content sections:**

- Game name and concept (original — not a Bomberman clone).
- Purpose and player experience.

- Player count (2 local players, same keyboard).
- Objective (capture cores, outscore opponent).
- Win / draw conditions.
- Player controls (P1 and P2 tables).
- Brief description of each screen (start, game, result).

**Acceptance criteria:** covers all RF-17.1 items. Readable without playing first.

---

## TASK-66 · docs/reglas.md

**Files to create:** docs/reglas.md

### Content sections:

- Arena description (15×11, cell types).
- Player attributes table (hp, energy, resources, score, bombAvailable, shieldActive, alive).
- Movement rules.
- Bomb placement and explosion rules (radius, chain reactions, max 1 damage per player).
- Core capture rules (no auto-capture, explicit action, +5/+2).
- Resource pickup rules (auto on move, two types).
- Special action rules (3-energy cost, offensive/defensive).
- Scoring table.
- Invalid actions table (all 10 error codes with human-readable explanation).
- Victory conditions (25 points, time limit, draw).
- Reactor Pulse event (timing, effect, one-time).
- Moving elements table (confirms RF requirement: players, bombs, cores, resources, pulse).

**Acceptance criteria:** covers all RF-17.2 items. A player can learn the game from this document alone.

---

## TASK-67 · docs/api.md

**Files to create:** docs/api.md

### Content sections:

- Base URL.
- `POST /api/game` — full spec, example request/response JSON.
- `GET /api/game/:gameId` — full spec, example response.
- `POST /api/game/:gameId/action` — full spec for all four action types, all 10 error codes with HTTP status.

- `POST /api/game/:gameId/test/force-end` — description, availability restriction, example.
- Full `GameState` JSON example (populated, not empty).
- `TestScenario` schema description.

**Acceptance criteria:** all endpoints fully documented with curl examples. A developer can call any endpoint from this document without reading source code.

---

## TASK-68 · `docs/decisiones.md`

**Files to create:** `docs/decisiones.md`

### Content sections:

- Polling vs WebSocket (why polling, tradeoffs, 500ms interval).
- State in Express memory (why no database, tradeoffs, restart caveat).
- Same keyboard two players (simplicity, exam scope).
- TypeScript on both sides (type safety, mirrored types).
- Vite proxy in dev / Express serves static in prod (same-domain requirement).
- No prohibited libraries (compliance table).
- `constants.ts` — single source of truth for tunable values.
- Environment variables table (all vars, default values, where used).
- Deployment platform choice (Railway, rationale).
- Timer management design (why `clearGameTimers` on restart).
- Test-only route restriction design (NODE\_ENV check, not imported in prod).
- Known risks and mitigations.
- Changes from original design (if any).

**Acceptance criteria:** covers all RF-17.4 items. Sufficient to explain every non-obvious technical choice during a 10-minute defence.

---

## TASK-69 · `docs/investigacion.md`

**Files to create:** `docs/investigacion.md`

### Content sections:

- Playwright research (what it is, selector strategy, `request` fixtures, headless vs headed, `wait-on` for CI).
- Deployment research (Railway platform, `railway.toml`, GitHub Actions integration, environment variable management).
- Docker note (not used — not needed since Railway uses Nixpacks; explain decision).

- AI usage log table:

| Prompt                   | Relevant response summary | What was incorporated | What the student verified | Post-AI modifications |
|--------------------------|---------------------------|-----------------------|---------------------------|-----------------------|
| (row per AI interaction) |                           |                       |                           |                       |

**Acceptance criteria:** covers all RF-17.5 items. AI usage table has at least one fully completed row.

---

## TASK-70 · README.md

**Files to create:** README.md

**Content sections:**

- Project name and one-line description.
- Prerequisites (Node 20, npm 9+).
- Installation (`npm install`).
- Development (how to start backend dev server and frontend dev server — manual, not `npm start`).
- Build (`npm run build`).
- Run production (`npm start` after build).
- E2E tests local (`npm run test:e2e`).
- E2E tests headed (`npm run test:e2e:headed`).
- E2E tests against production.
- Deployment (link to docs/decisiones.md, Railway URL).
- Public URL.
- Environment variables table.

**Acceptance criteria:** a person unfamiliar with the project can run it locally from the README alone.

---

## WAVE 17 — Final Audit

---

### TASK-71 · Magic number audit

**Objective:** Confirm that every tunable game constant is read from `constants.ts` and no magic numbers remain in game logic.

**Files to modify:** any file found to contain magic numbers.

**Dependencies:** all previous waves.

**Exact changes:**

Run:

```
grep -rn "\b120\b|\b25\b|\b3000\b|\b8000\b|\b4000\b|\b500\b|\b0.5\b" \
  backend/src/game/ --include="*.ts" | grep -v "constants.ts"
```

For each hit: replace the literal with the corresponding constant from `constants.ts`.

**Acceptance criteria:**

- No tunable game values appear as literals outside `constants.ts`.
- `cd backend && npx tsc --noEmit` exits 0 after changes.
- `cd backend && npm run lint` exits 0.

**Validation:**

```
grep -rn "[0-9]" backend/src/game/ --include="*.ts" | grep -v "constants.ts" | grep -v "///  
# manual review of remaining hits
```

**Expected result:** changing one constant in `constants.ts` changes the behaviour everywhere.

---

## TASK-72 · Prohibited library audit

**Objective:** Confirm no prohibited library is present in any `node_modules` or source file.

**Dependencies:** TASK-71.

**Exact changes:** none if clean; remove any violations found.

**Validation:**

```
grep -r  
"axios|react-router|redux|tailwind|bootstrap|socket.io|three.js|phaser|matter|cannon|b  
abylon" \  
  frontend/src backend/src --include="*.ts" --include="*.tsx"  
# must return empty
```

```
grep -E "\"axios\"|\"react-router\"|\"redux\"|\"tailwind\"|\"bootstrap\"|\"socket\" \" \
  frontend/package.json backend/package.json package.json
# must return empty
```

**Acceptance criteria:** all grep commands return empty.

**Expected result:** exam compliance confirmed.

---

## TASK-73 · Full exam rubric self-audit

**Objective:** Map every rubric criterion to evidence in the repository. Create [docs/rubrica.md](#).

**Files to create:** [docs/rubrica.md](#)

**Dependencies:** TASK-72.

**Exact changes:**

Create an audit table covering every rubric category. Template:

| # | Rubric criterion                | Evidence file / location                                       | Status |
|---|---------------------------------|----------------------------------------------------------------|--------|
| 1 | Rules and playable experience   | <a href="#">docs/reglas.md</a> , live game                     | ✓ / ✗  |
| 2 | Player interaction and movement | <a href="#">actions.ts</a> move, <a href="#">useKeyboard</a>   | ✓      |
| 3 | React + TypeScript              | <a href="#">frontend/src/**/*.tsx</a>                          | ✓      |
| 4 | Express + TypeScript            | <a href="#">backend/src/**/*.ts</a>                            | ✓      |
| 5 | HTTP REST integration           | <a href="#">routes/game.ts</a> , <a href="#">api/client.ts</a> | ✓      |
| 6 | GitHub Actions / lint           | <a href="#">.github/workflows/lint.yml</a>                     | ✓      |
| 7 | E2E tests                       | <a href="#">tests/e2e/*.spec.ts</a> , Actions                  | ✓      |
| 8 | Deployment                      | Railway URL, <a href="#">deploy.yml</a>                        | ✓      |
| 9 | Repository / documentation      | <a href="#">docs/</a> , <a href="#">README.md</a>              | ✓      |

Additional checklist:

- No prohibited libraries.
- No external game engine.
- Backend makes all significant game decisions.
- At least one GET and one POST endpoint implemented.
- Same domain and port in production.
- Public deployment responds correctly.
- Production E2E works against public URL.
- Local headed E2E works.
- Test-only functionality not accessible in production (verified in TASK-34).
- All timers cleaned up on restart (verified in TASK-50).
- No contradictory game rules.
- No unfinished TODOs or placeholder implementations.
- All 6 E2E tests pass in CI.

**Acceptance criteria:** every rubric row marked ✓ . Every checklist item checked.

**Expected result:** complete exam compliance evidence in one document.

---

## TASK-74 · Defence guide

**Objective:** Create a 10-minute oral defence script with answers to likely examiner questions.

**Files to create:** `docs/guia-defensa.md`

**Dependencies:** TASK-73.

**Content:**

10-minute script:

1. (1 min) Game concept and original rules — what makes it not a Bomberman clone.
2. (1 min) Architecture overview: Express + React, polling, single URL.
3. (1 min) `constants.ts` — live demo: change `BOMB_TIMER_S`, rebuild, show effect.
4. (2 min) Walk through `engine.ts` + `actions.ts` — backend makes all decisions.
5. (1 min) `bombs.ts` — `setTimeout` in Express, not React.
6. (1 min) `useKeyboard` + `usePolling` — no debounce, event.repeat filter.
7. (1 min) Three GitHub Actions workflows, production URL.
8. (1 min) E2E: local deterministic vs. production adaptive.
9. (1 min) Questions.

Likely examiner questions with prepared short answers:

- Why polling and not WebSocket?
- Why is the game state in Express memory and not a database?
- What prevents a player from receiving 2 damage from one explosion?
- What happens if both players capture the same core simultaneously?
- How does the Reactor Pulse event work?
- Why do the production E2E tests not use `force-end`?
- How do you change the game duration? (Answer: `GAME_DURATION_S` in `constants.ts`.)
- How do you change bomb radius? (Answer: `BOMB_RADIUS` in `constants.ts`.)
- How do you change victory score? (Answer: `VICTORY_SCORE` in `constants.ts`.)

**Acceptance criteria:**

- Script fits within 10 minutes when read aloud.
  - Every answer references actual code location.
- 

## Summary

**Total tasks: 74**

---

### Tasks by wave

| Wave                        | Tasks       | Count |
|-----------------------------|-------------|-------|
| 1 — Infrastructure          | TASK-01..06 | 6     |
| 2 — Types                   | TASK-07..08 | 2     |
| 3 — Store and factory       | TASK-09..15 | 7     |
| 4 — Engine                  | TASK-16..19 | 4     |
| 5 — Player actions          | TASK-20..24 | 5     |
| 6 — Bomb system             | TASK-25..27 | 3     |
| 7 — Cores, resources, pulse | TASK-28..30 | 3     |
| 8 — REST API                | TASK-31..35 | 5     |